

Weeks 7–8: Regularization & Activation Functions

Deep Learning – IITM BS
Notes with Intuition + Diagrams

Contents

1	Bias-Variance Tradeoff	2
2	Regularization: Fighting Overfitting	3
2.1	L2 Regularization (Weight Decay)	3
2.2	Data Augmentation	3
2.3	Parameter Sharing	3
2.4	Adding Noise (Input or Output)	3
2.5	Early Stopping	4
2.6	Ensemble Methods: Bagging	4
2.7	Dropout	4
3	Unsupervised Pre-training (Deep Learning Revival)	5
4	Activation Functions: The Complete Picture	5
4.1	Newer Activations: GELU and Swish	6
5	Practice Problems	7
6	Solutions	8

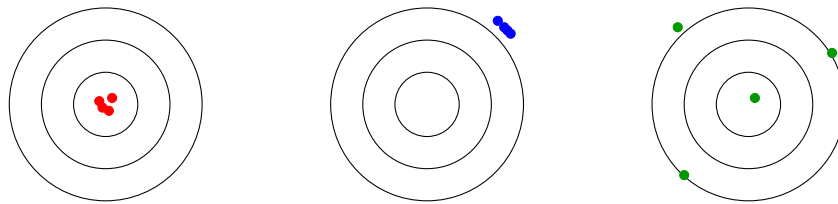
Bias-Variance Tradeoff

Intuition:

The archer analogy Imagine an archer shooting at a target repeatedly.

- **High bias:** All arrows land in a tight cluster, but far from the bullseye (consistently wrong). This is like a model too *simple* to capture the true pattern (underfitting).
- **High variance:** Arrows are scattered all over the place, sometimes near the bullseye, sometimes far. This is like a model too *sensitive* to the specific training data it happened to see (overfitting)—slightly different training data gives wildly different models.

Low Bias, Low Variance High Bias, Low Variance Low Bias, High Variance

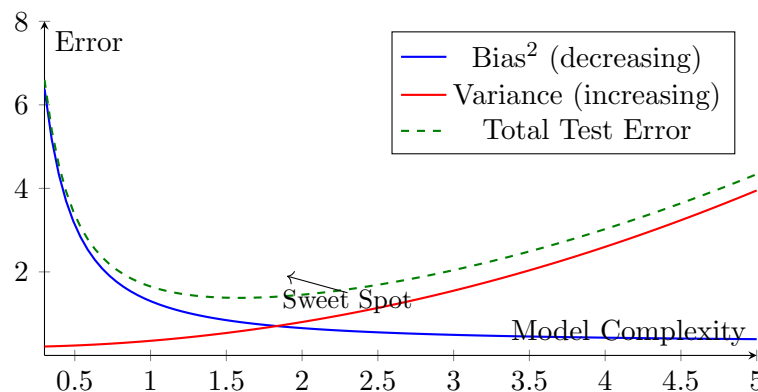


Formal Definitions

$$\text{Bias}[\hat{f}(x)] = E[\hat{f}(x)] - f(x) \quad \text{Variance}[\hat{f}(x)] = E[(\hat{f}(x) - E[\hat{f}(x)])^2]$$

$$E[(y - \hat{f}(x))^2] = \text{Bias}^2 + \text{Variance} + \sigma^2 \text{ (irreducible error)}$$

Simple models: high bias, low variance. **Complex models:** low bias, high variance.



Intuition:

Why the U-shape? Why not just make the model as complex as possible? As you add complexity (more layers, more parameters), the model gets better at *fitting the training data exactly*—bias drops. But eventually it starts memorizing noise and quirks specific to the training set rather than the true underlying pattern—variance shoots up. Since test error is the *sum* of both, there's a sweet spot in the middle. Since deep learning models are typically **over-parameterized** (billions of params, millions of samples), we live on the right side of this curve by default—which is exactly why regularization is essential.

2 Regularization: Fighting Overfitting

Intuition:

The core principle behind ALL regularization methods. Instead of minimizing just the training error, minimize (**training error** + $\Omega(\theta)$), where $\Omega(\theta)$ is a penalty that is **large for complex models** and **small for simple models**. This forces the optimizer to prefer simpler solutions unless the extra complexity is really "worth it" in reducing training error.

2.1 L2 Regularization (Weight Decay)

L2 Regularization

$$\bar{L}(w) = L(w) + \frac{\alpha}{2} \|w\|^2 \quad \nabla \bar{L}(w) = \nabla L(w) + \alpha w$$

Intuition:

Why does penalizing $\|w\|^2$ prevent overfitting? Overfit models often rely on a few weights growing very large to fit noise precisely (fitting sharp, wiggly curves through every data point). By adding a cost proportional to $\|w\|^2$, we discourage any single weight from growing too large, pushing all weights toward zero unless the data strongly justifies otherwise. The gradient update becomes $w \leftarrow w - \eta(\nabla L(w) + \alpha w) = (1 - \eta\alpha)w - \eta\nabla L(w)$ —literally shrinking w a little bit every step, hence "weight decay."

2.2 Data Augmentation

Intuition:

Free extra data from what you already have. If rotating, shifting, or slightly blurring an image of a cat still shows a cat, then these transformed versions are *valid new training examples*—for free! This effectively increases the size and diversity of your training set without collecting new data, making the model less able to memorize any single specific version of an image and forcing it to learn the underlying invariant pattern ("catness").

2.3 Parameter Sharing

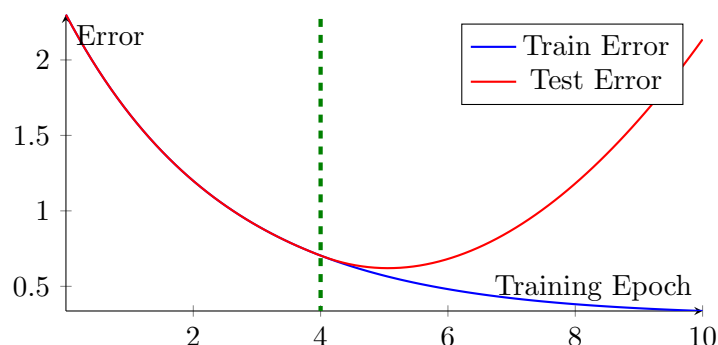
Same weights reused at different positions (e.g., CNN filters, RNN weights). Drastically reduces the number of *independent* parameters that could overfit, since the same weight must work well across many different inputs/positions simultaneously.

2.4 Adding Noise (Input or Output)

Intuition:

Why does noise help, rather than hurt? Adding small Gaussian noise to the input at every training step means the model never sees the *exact same* data point twice—it's forced to learn a function that is robust to small perturbations, rather than a function that passes exactly through every training point. In fact, for simple networks, this is mathematically **equivalent to L2 regularization!**

2.5 Early Stopping



Early Stopping Algorithm

Keep a **patience** parameter p . Monitor validation error after each epoch. If validation error doesn't improve for p consecutive epochs, **stop** and return the model from p steps earlier (when validation error was still improving).

2.6 Ensemble Methods: Bagging

Intuition:

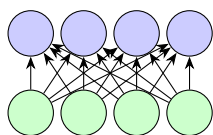
Wisdom of the crowd A single model's errors might be systematic (always wrong in the same way), but if you train *multiple independent* models (each on a different random sample of the data) and **average** their predictions, their individual errors tend to **cancel out**—as long as the errors are roughly independent/uncorrelated. This is exactly why ensembles of weak or noisy models often outperform any single strong model.

Key Caveat

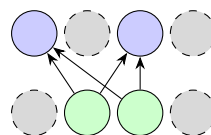
Bagging works well only if individual model errors are **uncorrelated**. If all models make the *same* mistakes (correlated errors), averaging doesn't help—the ensemble is only as good as a single model.

2.7 Dropout

Full Network



With Dropout (thinned)



Intuition:

Dropout = free, built-in ensemble At each training step, randomly "turn off" some neurons (with probability $1 - p$). This forces the remaining active neurons to work well *without relying on any specific set of teammates always being present*—preventing neurons from "co-adapting" (over-relying on each other in brittle ways). Effectively, you're training an enormous *ensemble* of different "thinned" sub-networks that all share weights, and averaging their effect—but without the huge cost of training separate networks.

Dropout Details

Typical: $p = 0.5$ for hidden units, $p = 0.8$ for visible/input units (probability of **keeping** the unit). Different random subset dropped each mini-batch. Only *active* parameters are updated for a given mini-batch.

3 Unsupervised Pre-training (Deep Learning Revival)

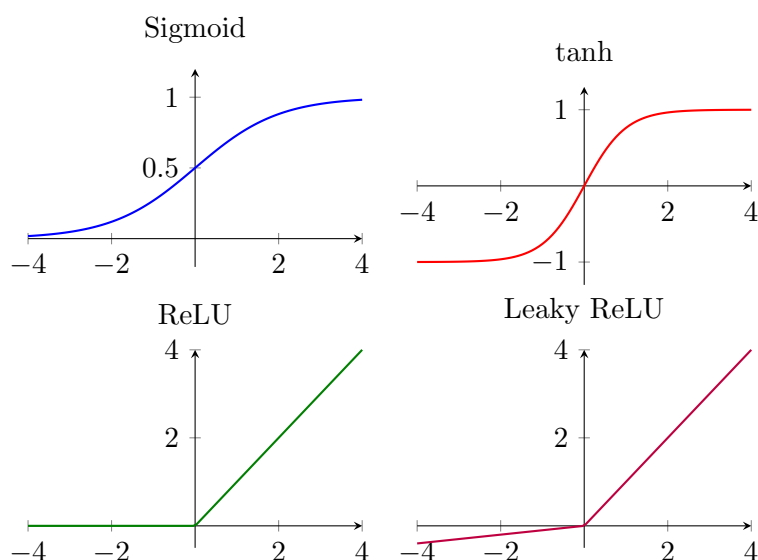
Intuition:

Warm-starting deep networks Before backprop worked reliably on deep networks (vanishing gradients made random-initialized deep nets hard to train), researchers found a clever workaround: train each layer *greedily*, one at a time, to reconstruct its own input (like a mini-autoencoder). This is unsupervised—no labels needed! After this layer-by-layer “warm-up,” the network’s weights start in a much better region of the loss landscape, and only then do we add the final output layer and fine-tune everything together with the supervised objective (backprop).

Benefits of Pretraining

- Better optimization (better initial weight region)
- Acts as a **regularizer** (representations learned aren’t overfit to labels)
- More robust to random initialization

4 Activation Functions: The Complete Picture



Intuition:

The evolutionary story of activation functions Sigmoid was the first choice (smooth, probability-like) but suffers from **saturation** (gradients vanish at extremes) and is **not zero-centered** (all outputs positive, causing zig-zag gradient updates). tanh fixes zero-centering but still saturates. ReLU is the breakthrough: it doesn’t saturate for positive inputs at all, is nearly free to compute, and converges much faster in practice. But it introduces a new problem: **dead neurons** (a neuron stuck outputting 0 forever, since its gradient is 0 for negative inputs). Leaky ReLU, PReLU, ELU are all attempts to keep ReLU’s benefits while patching the dead-neuron issue.

Full Comparison Table – EXAM CRITICAL

Function	Range	Zero-centered?	Saturates?	Dead neurons?	Cost
Sigmoid	$(0, 1)$	No	Yes	No	Expensive
tanh	$(-1, 1)$	Yes	Yes	No	Expensive
ReLU	$[0, \infty)$	No	No ($x > 0$)	Yes	Cheaper
Leaky ReLU	$(-\infty, \infty)$	$\approx$ Yes	No	No	Cheaper
Parametric ReLU	$(-\infty, \infty)$	$\approx$ Yes	No	No	Cheaper
ELU	$(-\alpha, \infty)$	$\approx$ Yes	No (neg. saturates softly)	No	Expensive

Derivatives – MEMORIZE

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)), \quad \max \text{ at } x = 0 : \sigma'(0) = 0.25$$

$$\tanh'(x) = 1 - \tanh^2(x), \quad \tanh'(0) = 1$$

$$\text{ReLU}'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$$

Why ReLU Neurons "Die"

If a large gradient update pushes a ReLU neuron's weights so that its pre-activation is *always negative* for every training example, its output is always 0, its gradient is always 0, and it can **never recover**—it's permanently dead. High learning rates make this more likely. Fix: initialize bias slightly positive, or use Leaky ReLU/ELU which have a small non-zero gradient for negative inputs.

4.1 Newer Activations: GELU and Swish**Modern Activations**

Swish: $f(x) = x \cdot \sigma(\beta x)$. When $\beta = 1.702$, approximates **GELU**. When $\beta = 1$, this is **SiLU** (Sigmoid-weighted Linear Unit).

GELU: $\text{GELU}(x) = x \cdot \Phi(x)$ where Φ is the Gaussian CDF—used in Transformers (BERT, GPT).

MaxOut: Take k affine transformations of the input, output the max. Generalizes ReLU and Leaky ReLU; a universal approximator with enough units.

5 Practice Problems

Problem 1: Bias-Variance Identification [2 marks, MCQ]

A model has very low training error but very high test error. This indicates:

- A. High bias, low variance B. Low bias, high variance C. High bias, high variance D. Low bias, low variance

Problem 2: L2 Regularization Gradient [2 marks]

$L(w) = w^2$, $\alpha = 0.5$. Compute $\nabla \bar{L}(w)$ at $w = 3$ using L2 regularization.

Problem 3: Weight Decay Update [2 marks]

Using the gradient from Problem 2, $\eta = 0.1$. Compute w_{new} .

Problem 4: Regularization Method Identification [2 marks, MSQ]

Training error 3%, test error 45%. Which methods help?

- A. L2 regularization B. Adding more layers C. Dropout D. Data augmentation E. Early stopping

Problem 5: Dropout Probability [2 marks]

A network has 100 hidden neurons, dropout $p = 0.5$ (keep probability). On average, how many neurons are active during one training step?

Problem 6: Activation Function Properties [3 marks, MSQ]

Which activation functions can suffer from vanishing gradients due to saturation?

- A. Sigmoid B. tanh C. ReLU (for $x > 0$) D. Leaky ReLU

Problem 7: Sigmoid vs tanh Derivative [2 marks]

Compute $\sigma'(0)$ and $\tanh'(0)$. Which has the larger maximum gradient?

Problem 8: Dead ReLU Diagnosis [2 marks, MCQ]

A ReLU neuron has weights such that its pre-activation is negative for all 10,000 training examples. What will happen during training?

- A. It will eventually recover B. It is permanently dead (zero gradient forever) C. It becomes a linear neuron D. Nothing unusual

6 Solutions

Problem 1: B. Low train error = low bias (fits data well). High test error = high variance (doesn't generalize) = overfitting.

Problem 2: $\nabla \bar{L}(w) = \nabla L(w) + \alpha w = 2w + \alpha w = 2(3) + 0.5(3) = 6 + 1.5 = \mathbf{7.5}$.

Problem 3: $w_{\text{new}} = 3 - 0.1(7.5) = 3 - 0.75 = \mathbf{2.25}$.

Problem 4: A, C, D, E. This is overfitting; more layers would worsen it.

Problem 5: On average, $100 \times 0.5 = \mathbf{50}$ neurons active.

Problem 6: A, B. Sigmoid and tanh saturate at both extremes. ReLU for $x > 0$ has constant gradient 1 (no saturation). Leaky ReLU never saturates.

Problem 7: $\sigma'(0) = 0.25$, $\tanh'(0) = 1$. **tanh has the larger maximum gradient** (4x larger), which is one reason it often trains faster than sigmoid despite still saturating.

Problem 8: B. If pre-activation is always negative, ReLU output is always 0, gradient is always 0, so no weight update ever occurs for this neuron—it's permanently "dead."